

Perception-Open CV



Task-I

Input

Output :

Using Python and OpenCV, write a program that:

- Loads the given image
- Parses the provided `boxes` data
- Draws bounding boxes over the cones in the image

Each entry in `boxes` resembles the following format :

```
[class_id, [x_center, y_center, width, height], confidence]
```

Where -

- `class_id` → type of cone (blue / yellow / orange etc.)
- `x_center`, `y_center` → center of bounding box, normalized (0-1)
- `width`, `height` → box size, normalized (0-1)
- `confidence` → detection confidence

Normalized means these values are fractions of image width/height, not pixels.



Code :

task_01.py > ...

```
1 import cv2
2 image = cv2.imread(r"C:\Users\Harshita Sinha\Downloads")
3 boxes = [
4     [2.0, [0.43476563692092896, 0.7194444537162781, 0.03984374925494194, 0.1111111119389534], 0.8438236117362976],
5     [2.0, [0.47539061307907104, 0.737500011920929, 0.04296875, 0.11666666716337204], 0.8504939675331116],
6     [0.0, [0.33671873807907104, 0.71111111283302307, 0.03593749925494194, 0.09444444626569748], 0.9057255387306213],
7     [3.0, [0.2679687440395355, 0.7361111044883728, 0.04374999925494194, 0.125], 0.919158935546875],
8     [0.0, [0.21054688096046448, 0.7479166388511658, 0.04921875149011612, 0.12638889253139496], 0.9196999073028564],
9     [3.0, [0.38945311307907104, 0.7201389074325562, 0.03984374925494194, 0.10972221940755844], 0.9205850958824158],
10    [3.0, [0.5414062738418579, 0.762499988079071, 0.05312500149011612, 0.14444445073604584], 0.9242506623268127],
11    [0.0, [0.610156238079071, 0.7909722328186035, 0.06406249850988388, 0.1736111044883728], 0.9282561540603638]
12 ]
13 width = 1280
14 height = 720
15 for class_id, bbox, confidence in boxes:
16     x_centre, y_centre, w, h = bbox
17
18     if(class_id == 0.0):
19         c = (255, 0, 0)
20     elif(class_id == 3.0):
21         c = (0, 255, 255)
22     elif(class_id == 2.0):
23         c = (0, 140, 255)
24
25     x_centre_px = int(x_centre*width)
26     y_centre_px = int(y_centre*height)
27
28     w_px = int(w*width)
29     h_px = int(h*height)
30
31     x1 = int(x_centre_px - w_px/2)
32     y1 = int(y_centre_px - h_px/2)
33
34     x2 = int(x_centre_px + w_px/2)
35     y2 = int(y_centre_px + h_px/2)
36
37     cv2.rectangle(image, (x1, y1), (x2, y2), c, 1)
```

normalised values

convert them to pixels

find coords of top left
& bottom right (pixels)

Draw boxes

(converting to pixels)

function to draw boxes using
opencv

```
38
39 ho = 25 cm
40 f = 1000 cm
41 hi = h*height
42 d = (f*ho)/hi
43 depth_text = f"{d:.2f} cm"
44
45 cv2.putText(image, depth_text, (x1, y1-5), cv2.FONT_HERSHEY_COMPLEX, 0.3, c, 1)
46 cv2.imshow("Loaded Image", image)
47 cv2.waitKey(0)
48 cv2.destroyAllWindows()
49 cv2.imwrite("output.png", image)
```

converting height of cones in the image
in to pixels

function to print the
value of the depth above each
box

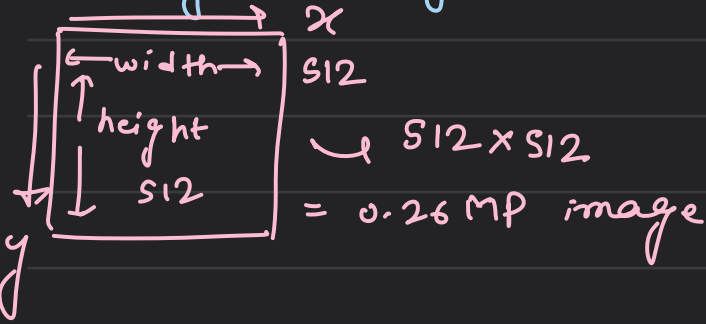
A Digital Image ($W \times H$)

* Numeric representation in 2D

* Referred to as $I(x, y)$ in

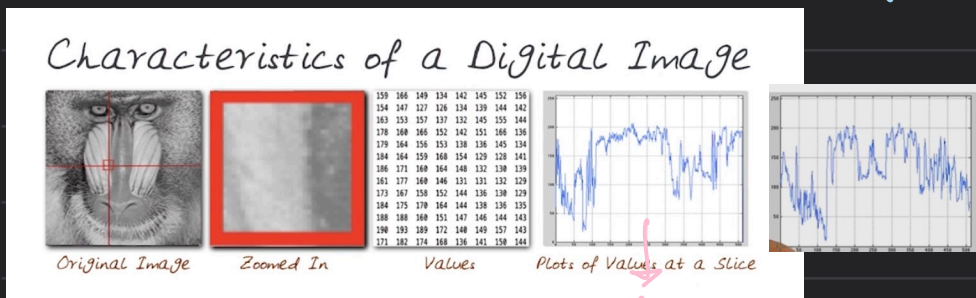
cont's fcn from, $I(i, j)$ in discrete

* Image resolution: expressed in terms of width & height of the image



Pixels: A 'picture element' that contains the light intensity at some location (i, j) in the image, $I(i, j)$
 ↳ Some Numeric value

Characteristics of a Digital Image:-



pixel vs I along x
 ↳ $\max I$
 255 - white
 0 - black
 ↳ $\min I$

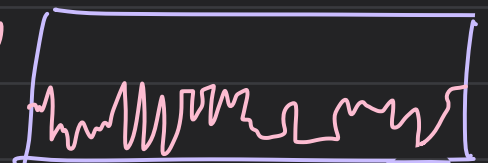
pixel vs I along y

- * A 2D array of pixels and respective intensities
- * image can be represented as matrix

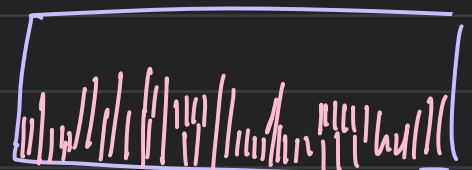
Digital image as a fcn:-



cont's signal



Discrete signal



• Typically, the fcn operation req. discrete values

• Sample the 2D space on a regular grid

• Quantization (conversion of continuous values to discrete values)

Quantize each sample (rounded to nearest integer)

- Matrix of integer values (Range: 0-255)

Digital Image Statistics

- Image statistics - avg., median, mode
- Scope - entire image or smaller region
- Histogram - distribution of pixel intensities in the image
- can be separate for each channel or region-based too.

Colour Digital Image: each colour image has 3 channels with their own color intensities



blended together

makes 3D data structure of
size: width x Height x channel

- each pixel has 3 intensities: Red (R), Green (G), Blue (B)
- each channel: 2^8 : 0-255, 24 bit colour channel.

Digital Image Formats:

- Raster image formats store a series of coloured dots
- Number of bits for each pixel represents "pixels".
the depth of colour.
- 1 bit: $2^1 \rightarrow 2$ colours (black or white)
- 4 bit: 16 colours, 8 bit per pixel: 256 different colours per pixel
- Images can also be 16, 24, 32 bits per pixel

- 24 bits per pixel usually means 8 bits per colour & there are 3 colours
- At the two highest levels, the pixels themselves can carry up to $16 \ggg 2^{16}$ different colours.

Common raster image formats: GIF, PNG, JPG, etc

Read & write images:

In python / openCV:

```

> import cv2
> image = cv2.imread('image.png')
> cv2.imwrite('output.png', image)

```

In matlab:

```

> image = imread('input.png')
> imwrite('output.png', image)

```

other functions to consider:

```

> imshow(image)
> imageinfo('input.png')
> help images

```

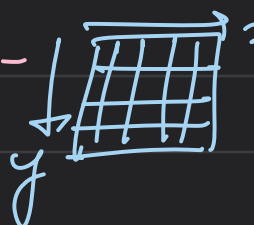
Read image, then find out:

- Dimensions of the image
- no. of color channels
- Bits per pixel.

```

> print image.shape % (w, h, # channels)
> print image.dtype % NumPy types

```

Digital image as a function:-  → Convert to discrete values

- How to obtain discrete values?
- Sample the 2D space on a regular grid
- Quantize each sample (rounded to "nearest integer")

Matrix of integer values (Range: 0 - 255)

Point-Process: Pixel/Point Arithmetic

8 bit per pixel

120	140	142
121	144	147
125	121	144

 $+$

120	140	142
121	144	10
125	80	0

 $=$

240	280	284
242	288	157
250	201	144

we will make the values greater than 255 to 255 & to the corresponding change in all the cells.

values greater than 255

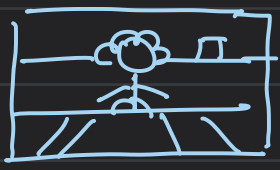
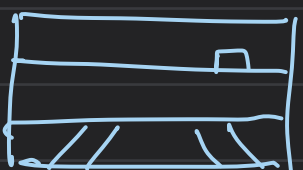
207	247	251
204	255	124
217	168	111

288 → 255

$\downarrow$
-33

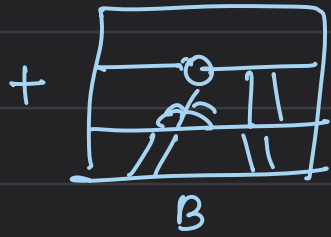
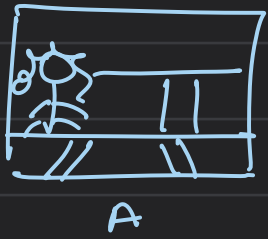
for subtraction we may get -ve values → must -ve to zero & add that no. to all other cells.

Pixel / Point Arithmetic: An example:


 $-$


= only the body of the person & every thing else becomes dark.

Pixel Operation : An example:



sum $\rightarrow > 255 \rightarrow 255$
 $\downarrow$
 close to white

= all the people will be present on the image will be as bright as white = $A+B+C$

We can add the 3 by multiplying by some weights

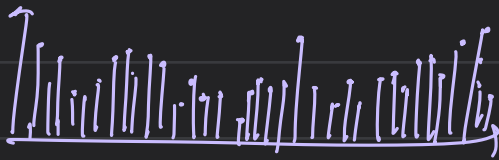
$\hookrightarrow aA + bB + cC$ $\hookrightarrow$ the resultant image's background will be similar to A, B or C but the people will fade

lecture - 82 to 85

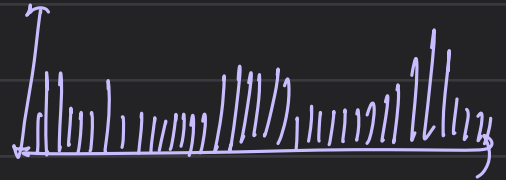
From Pixel / Point Operations to Groups of Pixels

How to Smooth a Signal?

consider a 1-D signal



process



method: Determine the avg of nbd values

moving avg: $\frac{1+1+1+1+1}{5}$

weighted avg:

$\frac{1+4+6+4+1}{16}$

Smoothing Process over an Image using Averages

move a 3×3 nbd and calculate the avg Δ replace the centre by this avg.

a more smoothed output

input (i,j)

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	0
0	0	0	90	0	90	90	0
0	0	0	90	90	90	90	0
0	0	0	90	90	90	90	0
0	0	0	90	90	90	90	0
0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

9x9

output (i,j)

	0	10	20	30	30	20	10
0	20	30	20	50	60	40	20
0	30	50	60	80	90	60	30
0	30	60	80	80	90	60	30
10	30	60	90	90	90	60	30
10	30	40	60	60	50	40	20
10	20	20	30	30	30	20	10
10	10	0	0	0	0	0	0

Smoothing Process over an Image using Averages

input (i,j)

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	0
0	0	0	90	0	90	90	0
0	0	0	90	90	90	90	0
0	0	0	90	90	90	90	0
0	0	0	90	90	90	90	0
0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

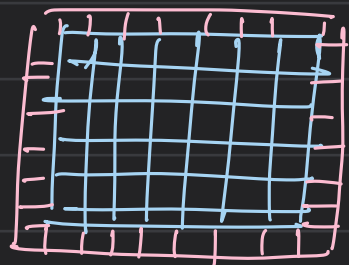
output (i,j)

0	10	20	30	30	20	10	
0	20	30	50	60	40	20	
0	30	50	60	80	90	60	
0	30	60	80	80	90	60	
10	30	60	90	90	90	60	
10	30	40	60	60	50	40	
10	20	20	30	30	30	20	
10	10	0	0	0	0	0	
0	0	0	0	0	0	0	
0	0	0	0	0	0	0	

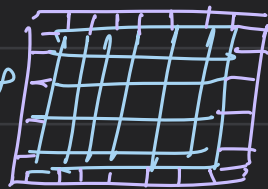
Results in Shades of Gray

Smoothing Process for the Edges of an Image

- we have to pad the edges with additional info
- a good way is to copy the edges columns again in the input image.



padded input



complete output

rows of padding
rep:

matrix: $n \times n$

: $\left\lceil \frac{n}{2} \right\rceil$ $\rightarrow$ no of
extra
columns

to be added

Some options for padding:

- $\rightarrow$ wrap around
- $\rightarrow$ reflect across edges
- $\rightarrow$ copy edge

Some observation : rubbing of small image (3×3) over a larger image (9×9)

* the small image "rubbed" is called the "Kernel"

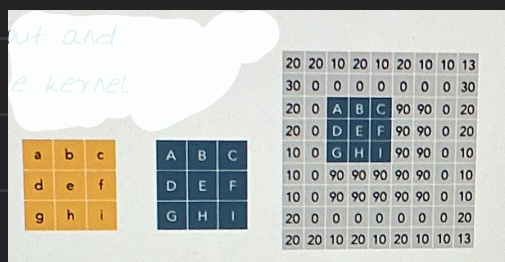
Kernel (i, j)

* nbd is $K = 1$: $\left\lceil \frac{n}{2} \right\rceil$ $\rightarrow$ window size: $2K + 1$

let refer to $F[i, j]$ as input & $G[i, j]$ as output & Kernel: as $h[i, j]$

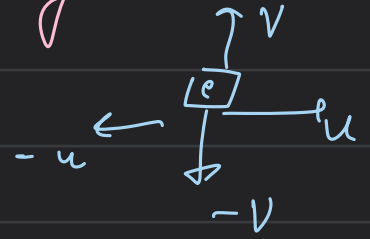
$$G[3, 3] = (A * a + B * b + C * c + D * d + E * e + F * f + G * g + H * h + I * i)$$

$$= \frac{1}{9} (A + B + C + D + E + F + G + H + I)$$



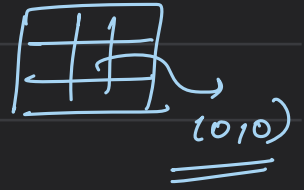
Mathematical Representation for Smoothing:

$$G[i,j] = \frac{1}{(2k+1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v]$$



more generally:

$$G[i,j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u,v] F[i+u, j+v]$$

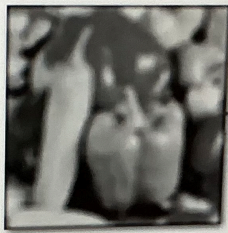


Atribute
non-uniform weights

this method is called Cross-Correlation (X-Correlation)

Example: Box Filter (Avg) for Smoothing

Example: Box Filter (Averaging!) for Smoothing



Box/Average Filter - 21 x 21

$$K[i,j] : 21 \times 21, F[i,j] : 512 \times 512$$

smoothing of the matrix

Special Case: Median Filtering:

20	20	1
20	0	0
20	0	90

median : 20, avg : 19

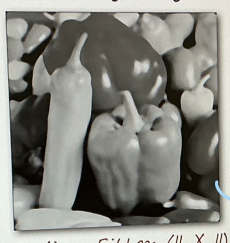
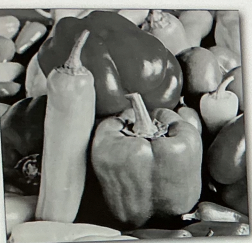
* Apply median filter to all 3x3 region over the whole image

non-linear operation & used in image processing.

* reduces noise

* preserves edges (sharp edges)

sharper edges



Median Filter (11 x 11)

'Salt & Pepper' Noise : Solve the problem as median filter

Cross-Correlation method:

In signal processing it is a measure of similarity of two wave-forms as a fn of time-lag applied to one of them.

* Also known as a sliding dot product or sliding inner product.

$$G = h \otimes F$$

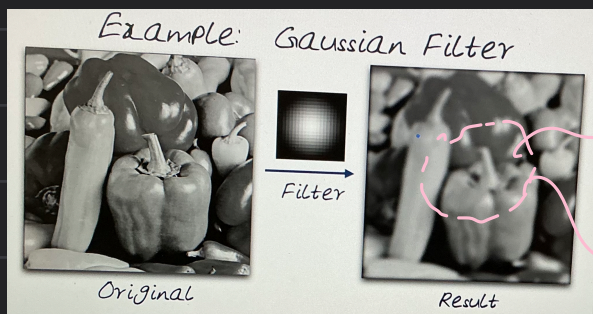
kernel input output

* Filtering an image:

replace each pixel with a LC of its neighbours with $h[u,v]$ which is a prescription for weights in the LC

eg: Box filter: flat kernel: $a=b=c=d=e=f=g=h$

eg: Gaussian filter: Values of kernel: Gaussian or Normal distribution



the centre is multiplied with higher weights of the kernel
sharper.

* when we inc the σ (variance) more the image becomes blur.

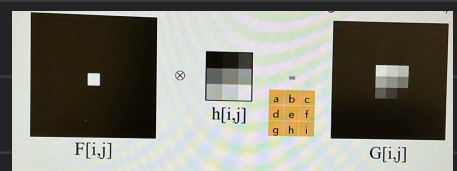
Filtering by a kernel (defining Convolution)

impulse image

$$F[i,j] \otimes \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = \begin{bmatrix} \dots & i & h & g & \dots \\ \dots & t & e & d & \dots \\ \dots & c & b & a & \dots \end{bmatrix}$$

flip in both x & y axis

* Result in a reversed response



Convolution Method:

- * Convolution is a mathematical operation on two fn F & h
- * Produces a 3rd fn that is typically viewed as a modified version
- * Gives the area of the overlap b/w the 2 fn
- * In a form of the amount that one of the original function is translated

$$G[i,j] = \sum_{u=-K}^K \sum_{v=-K}^K h[u,v] F[i-u, j-v]$$

$$G = h * F$$

- * Flip filter in both dimension
 - * Bottom to top
 - * Right to left } for $h[i,j]$
 - * & then apply cross-correlation
- it is like doing cross-correlation using a flipped kernel

Properties of Convolution

- * Linear & shifted invariants
- * Behaves the same everywhere (ie. the value of the output depends on the pattern in the image nbd, not the position of the nbd.)
- * Commutative: $F * G = G * F$
- * Associated: $(F * G) * H = F * (G * H)$
- * Identity : eg: Unit Impulse
 $\hookrightarrow E = [\dots 0, 0, 1, 0, 0 \dots] \hookrightarrow F * E = F$
- * Seperable: If the filter is seperable, convolve all rows & then convolve all columns.

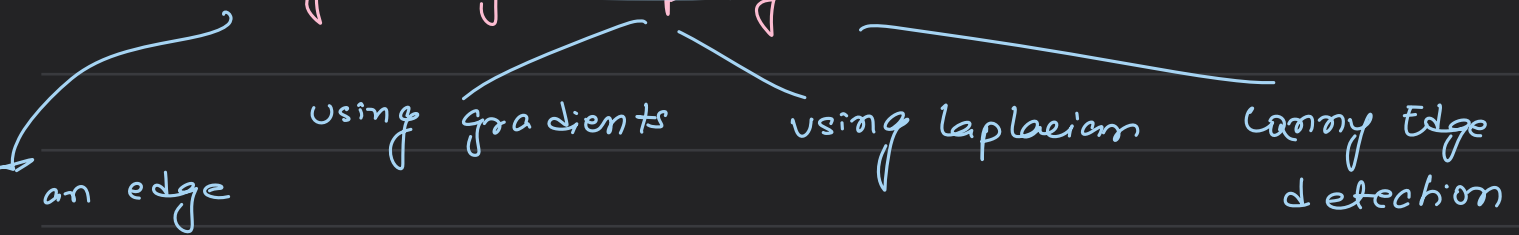
Linear filters: kernel ex.

eg: $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \times 2 = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$

Lecture — 114 to 123

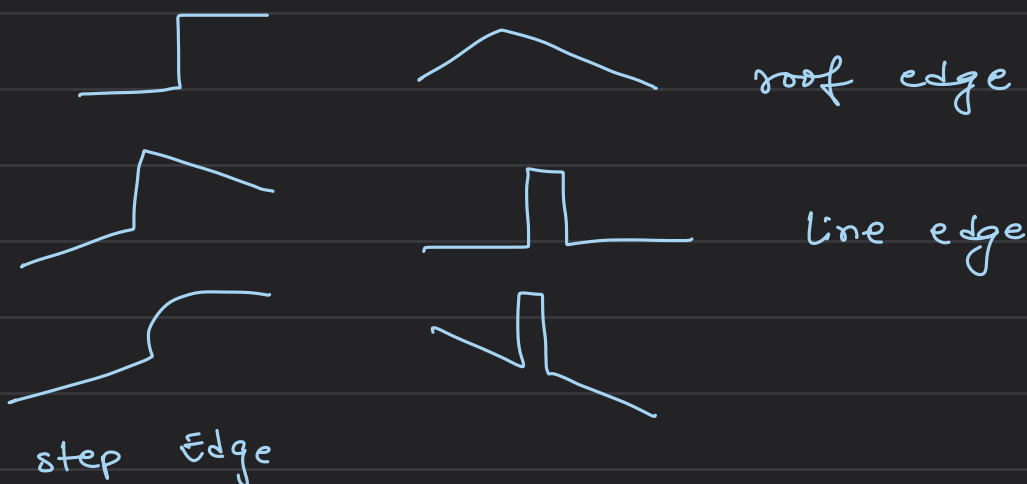
Edge Detection

Convert a 2D image into a set of points where image intensity changes rapidly.



Rapid changes in image intensity are caused by various physical phenomena

- ▷ Surface Normal Discontinuity
- ▷ Depth discontinuity
- ▷ Surface reflectance Discontinuity (diff material)
- ▷ Illumination Discontinuity (shadows)



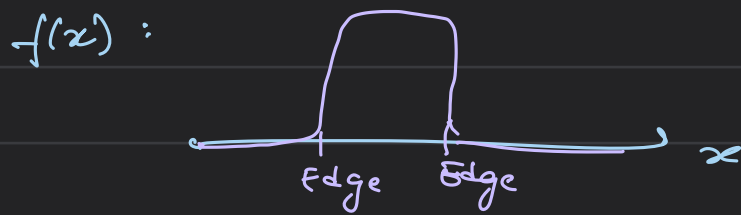
We want the edge operator that produces:

- Edge position (where in the pixel)
- Edge Magnitude (Strength)
- Edge orientation (direction)

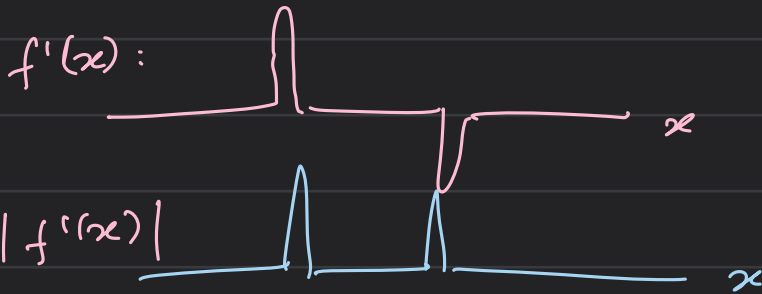
...value on pixels

Edge detection using gradients

1D Edge detection: an edge is a rapid change in image intensity in a small region



gradient represents the amount of change in the fcn



location of the peaks gives the position of the edge
 & height gives the magnitude of the edge

2D Edge detection:

* partial derivatives of a 2D cont^s fcn represents the amount of change along each dimension

Gradient (∇): $\nabla V = \frac{\partial V}{\partial x} \hat{i} + \frac{\partial V}{\partial y} \hat{j}$ represent the direction of most

rapid change in intensity

$$\|\nabla V\| = \left(\left(\frac{\partial V}{\partial x} \right)^2 + \left(\frac{\partial V}{\partial y} \right)^2 \right)^{1/2}$$

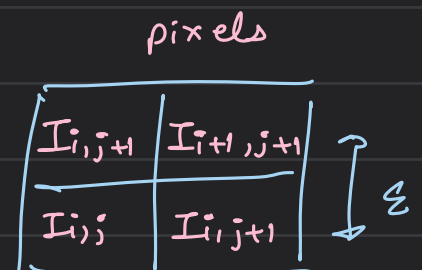
$$\theta = \tan^{-1} \left(\frac{\partial V / \partial y}{\partial V / \partial x} \right)$$

Discrete gradient operator (∇)

finite diff. approximations:

$$\frac{\partial I}{\partial x} \approx \frac{1}{2\epsilon} \left((I_{j+1,j+1} - I_{i,j+1}) + (I_{i+1,j} - I_{i,j}) \right)$$

$$\frac{\partial I}{\partial y} \approx \frac{1}{2\epsilon} \left((I_{i+1,j+1} - I_{i+1,j}) + (I_{i,j+1} - I_{i,j}) \right)$$



this can be implemented using convolution

$$\frac{\partial}{\partial x} \approx \frac{1}{2\epsilon} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$\frac{\partial}{\partial y} \approx \frac{1}{2\epsilon} \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

⊗ convolution
flips have been
applied

Comparing Gradient (∇) Operators				
Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -2 & -3 & 0 & 3 & 2 \\ -3 & -5 & 0 & 5 & 3 \\ -2 & -3 & 0 & 3 & 2 \\ -1 & -2 & 0 & 2 & 1 \end{bmatrix}$
$\frac{\partial I}{\partial y}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 3 & 5 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ -2 & -3 & -5 & -3 & -2 \\ -1 & -2 & -3 & -2 & -1 \end{bmatrix}$

Good localisation → Poor localisation

Noise Sensitive
(less pixels)

less Noise
sensitive

Poor Detection

Good
Detection

Edge Thresholding:

Std: (single Threshold T)

$\|\nabla I(x,y)\| < T$: Definitely Not an edge

$\|\nabla I(x,y)\| \geq T$: Definitely an edge

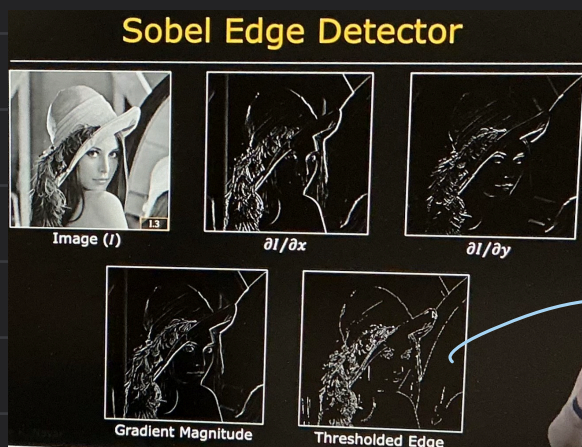
T : limiting case in order
to be considered as an edge

Hysteresis Based: (Two Thresholds $T_0 < T_1$)

$\|\nabla I(x,y)\| < T_0$: Definitely not an edge

$T_0 \leq \|\nabla I(x,y)\| < T$: is an edge if nbd pixel is an edge

$\|\nabla I(x,y)\| \geq T$: Definitely an edge



more no. of pixels are considered
as an edge.